

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Branch Coverage Report

Blood Bank Management System

Version 1.0

Prepared by Team 9 — Ramith M R (PES1UG24CS367)

DevOps / Configuration & Release Lead

F-Section, PES University

07/11/2026

Tested commit: 61d576d (main)

Revision History

Version	Date	Details
1.0	07/11/2026	Initial branch coverage report for BBMS at commit 61d576d.

Summary

Item	Result
Tested commit	61d576d — Merge pull request #50 (E5-S3 reports and history), 01/11/2026
Test run	131 tests run in 303.6 s — OK (skipped=2). 129 passed, 0 failed, 2 skipped.
Overall branch coverage	81.25% (117 of 144 branch outcomes), with 17 partially covered decisions
Overall statement coverage (same run)	89.47% (756 of 845 statements), matching the Code Coverage Report
Apps at 100% branch coverage	accounts (4 of 4), reports (18 of 18)
Apps below 80% branch coverage	donations 28.6% (4 of 14); inventory 50.0% (2 of 4). The STP sets no branch target, so these are reported, not failed.
Other apps	blood_requests 88.6%; blood_search 87.5%; donors 83.3%; issues 82.4% (93.3% excluding seed_perf_data.py); monitoring has no branches (N/A)

1. Objective and Scope

1.1 Objective

This report measures how many decision outcomes (branches) in the BBMS application code are exercised by the automated Django test suite. Each if, elif, for and while decision has two outcomes, and both should be executed. Branch coverage is measured with coverage.py branch tracking (--branch).

The Software Test Plan (STP v1.0, Section 14) sets an 80% statement coverage target per app and does not set a numeric branch target. This report therefore

measures and discloses branch coverage without assigning a pass or fail threshold, and refers to the 80% statement target only for comparison.

1.2 Scope

Included: all nine BBMS application packages — accounts, donors, donations, inventory, monitoring, blood_search, blood_requests, issues and reports.

Excluded: database migrations (*/migrations/*), test files (*/tests.py, */test_*.py), the Django project settings package (config), the virtual environment and third-party libraries. Statement coverage for the same run is reported only for reference; full statement results are in the Code Coverage Report.

1.3 Reference Documents

- BBMS Software Requirements Specification (SRS), Version 1.0.
- BBMS Software Architecture and Design Specification (SAD), Version 1.0.
- BBMS Software Test Plan (STP), Version 1.0 — Sections 5 and 14.
- BBMS Unit Test Report, Version 1.0 (commit 61d576d).
- BBMS Code Coverage Report, Version 1.0 (commit 61d576d).
- BBMS source code, Git revision 61d576d4caa3d6fac55a9ccd4205509ae8bfe1cd.

2. Test Environment

Parameter	Details
Operating system	Windows, PowerShell (command-line run from the repository root)
Python	3.12 (virtual environment bbms-py312)
Django	5.2.18
Coverage tool	coverage.py 7.16.2 (C extension), branch tracking enabled (--branch)
Test runner	Django test runner (manage.py test), Django TestCase
Database	SQLite test database created and destroyed by Django for the run (USE_SQLITE=True)
Source revision	61d576d4caa3d6fac55a9ccd4205509ae8bfe1cd (GitHub main)
Run date	07/11/2026

Parameter	Details
Settings	DEBUG=False, ALLOWED_HOSTS=127.0.0.1,localhost,testserver, local test secret key

No real donor or patient data was used. Django created a temporary test database, applied all migrations and removed it after the run.

The Code Coverage Report was produced with Python 3.13.16. This run used Python 3.12, which matches the team's CI workflow. Statement totals are identical in both runs (845 statements, 89 missed), which confirms that both reports measured the same code. The branch run took 303.6 s, compared with 103.0 s for the statement run, because branch tracing slows execution.

3. Test Method

3.1 Commands

The full automated suite was run under coverage.py with branch tracking and the scope given in Section 1.2. The reports were then generated:

```
python -m coverage erase
python -m coverage run --branch --
source=accounts,donors,donations,inventory,monitoring
,
  blood_search,blood_requests,issues,reports --
omit="*/migrations/*,*/tests.py,*/test_*.py"
manage.py test -v 2
python -m coverage report -m
python -m coverage json -o
Docs/Testing/Evidence/BranchCoverage/branch_coverage.
json
python -m coverage html -d htmlcov
```

3.2 Calculation

Branch coverage = covered branch outcomes ÷ total branch outcomes × 100. For each app, covered_branches and num_branches are summed over every measured file of that app in the coverage JSON, and the sums are divided. The overall figure is the sum over all apps. Zero-branch apps are reported as N/A, not 100%.

Notation. An arc A→B means the jump from line A to line B never happened, so one outcome of the decision on line A was never tested. For example,

issues/issuance.py 85→83 means the branch where a batch has no units left was never taken.

3.3 The 88% Figure in the HTML Report

The HTML index title ("Coverage report: 88%") and the TOTAL row of the terminal report combine statements and branches: (756 + 117) ÷ (845 + 144) = 873 ÷ 989 = 88.3%. This combined score is not branch coverage and is not used in this report.

3.4 Pass Criterion

The STP sets no branch coverage target, so no pass or fail is assigned to branch coverage in this report. Findings are reported as measured.

4. Results

4.1 Test Run

The branch-instrumented run executed 131 tests in 303.6 s — OK (skipped=2). No test failed. The two skipped tests are the PostgreSQL-only concurrent-issue test and the browser portability test, the same two skipped in the statement run.

4.2 Branch Coverage by Django App

Django app	Covered branches	Total branches	Branch %	Partial decisions
accounts	4	4	100.0%	0
blood_requests	39	44	88.6%	5
blood_search	7	8	87.5%	1
donations	4	14	28.6%	4
donors	15	18	83.3%	3
inventory	2	4	50.0%	2
issues	28	34	82.4%	2
monitoring	0	0	N/A	0
reports	18	18	100.0%	0
Total (all 9 apps)	**117**	**144**	**81.25%**	**17**

Notes: monitoring contains no branch decisions, so it is N/A. The issues app includes issues/management/commands/seed_perf_data.py, which

contributes 4 branches, none executed. Excluding that file, issues is 28 of 30 branches (93.3%).

4.3 Files with Missing Branch Outcomes

The table lists every file with at least one branch outcome that was never executed. Every other measured file was fully covered for branches.

File	Branches	Missing outcomes (A→B)	Meaning of the missing outcome
blood_requests/permissions.py	10	22→23, 25→26, 61→62	Anonymous user in user_has_role; superuser shortcut in user_has_role; anonymous user in can_review_request
blood_requests/views.py	30	53→56, 112→144	Submit page opened without a blood_group value; review POST with a valid decision but an invalid form
blood_search/views.py	8	23→24	Superuser shortcut in the availability search
donations/forms.py	2	38→39	Expiry date earlier than the donation date
donations/models.py	2	24→25	Donation dated in the future fails validation
donations/views.py	10	24→27, 40→53, 53→55, 53→66, 55→56, 55→58, 60→53, 60→61	Non-staff user gets 403; invalid donation form never submitted, so the form-error code never runs
donors/views.py	18	73→78, 94→95, 138→156	Search without a phone number; non-staff user opens donor history; invalid donor update form
inventory/	2	28→29	Invalid blood group

File	Branches	Missing outcomes (A→B)	Meaning of the missing outcome
services.py			returns 0 units
inventory/views.py	2	22→26	User without a BBMS role gets 403
issues/issuance.py	12	85→83, 107→108	Batch with no remaining units is skipped; break after exact allocation is never reached
issues/ management/ commands/ seed_perf_data.py	4	40→41, 40→48, 128→129, 128→144	Whole command never executed (tooling file)

5. Findings and Gaps

5.1 Apps Below 80% Branch Coverage

donations (28.6%) and inventory (50.0%) have the lowest branch coverage. Nearly all of the donations gaps are negative paths: a non-staff user is refused, and a form is submitted with invalid data. The inventory gaps are an invalid blood group and a user with no BBMS role. These are the same kinds of paths the statement report found untested, now shown at decision level.

5.2 Possible Defect for Team Review

In blood_requests/views.py, arc 112→144 goes straight to the final redirect when the review decision is valid but the form is invalid. As far as the code shows, no error message is added on that path, so the reviewer receives no feedback. This report does not change the code. The team should confirm the behaviour with a test and decide whether to fix it in a separate change.

5.3 Gaps and Proposed Tests

The tests below are proposed for a separate change. They are not written in this branch.

Location	Missing outcome	Proposed test
donations/ views.py:24 (24→27)	Non-staff user records a donation	Log in as Hospital Staff and POST a donation; expect PermissionDenied and no

Location	Missing outcome	Proposed test
		Donation saved.
donations/ views.py:40 (40→53 and form-error block)	Invalid donation form	POST units = 0 as Blood Bank Staff; expect form errors shown as messages and nothing saved. One test covers seven of the eight arcs in this file.
donations/ forms.py:38 (38→39)	Expiry date before donation date	Submit the form with an expiry date earlier than the donation date; expect an error on expiry_date.
donations/ models.py:24 (24→25)	Donation dated in the future	Call full_clean() on a donation dated tomorrow; expect ValidationError.
donors/views.py:73 (73→78)	Search without a phone number	GET the search page with a blank phone number; expect the page to render with no results.
donors/views.py:94 (94→95)	Non-staff user views donor history	Log in as Hospital Staff and open donor history; expect PermissionDenied.
donors/views.py:138 (138→156)	Invalid donor update	POST the update form with a required field blank; expect the form to be redisplayed and nothing saved.
blood_requests/ views.py:53 (53→56)	Submit page without blood group	GET the submit page with no blood_group parameter; expect an empty form.
blood_requests/ views.py:112 (112→144)	Valid decision, invalid review form	POST decision=approve with an invalid note; expect an error message and no status change. See Section 5.2.
blood_requests/ permissions.py:22 (22→23)	Anonymous user in user_has_role	Call user_has_role with AnonymousUser; expect False.

Location	Missing outcome	Proposed test
blood_requests/ permissions.py:25 (25→26)	Superuser in user_has_role	Call user_has_role with a superuser; expect True.
blood_requests/ permissions.py:61 (61→62)	Anonymous user in can_review_request	Call can_review_request with AnonymousUser; expect False.
blood_search/ views.py:23 (23→24)	Superuser in availability search	Superuser opens the availability search; expect access.
inventory/ services.py:28 (28→29)	Invalid blood group	get_available_units("XYZ") returns 0.
inventory/ views.py:22 (22→26)	User without a BBMS role	A user with no group opens the inventory dashboard; expect 403.
issues/issuance.py:85 (85→83)	Batch with no remaining units	Issue all units of one batch, then issue again; the exhausted batch must be skipped.
issues/ issuance.py:107 (107→108)	break after exact allocation	Request 2 units from batches of 2 and 3 units; expect allocation from the first batch only and the loop to break on the second.
issues/ management/ commands/ seed_perf_data.py	Command never executed	Tooling file. Team decision: add a small smoke test, or record it in the Test Plan as tooling excluded from coverage.

5.4 Measurement Notes

- Date and near-expiry rules (the 7-day window and expired stock) are database filters, not Python branch decisions, so branch coverage does not measure them. They are covered by the Unit Test Report.

- coverage.py does not count boolean expressions (for example profile is not None and profile.role in profile_roles) or comprehension filters as separate branch outcomes.
- seed_perf_data.py is included because the measurement scope follows the guide, which excludes only migrations and test files. It lowers issues from 93.3% to 82.4%.

5.5 Limitations

- The PostgreSQL-only concurrent-issue test was skipped, so the row-locking path that runs only under PostgreSQL is not measured.
- The browser portability test was skipped (it needs BROWSER_TESTS and Playwright).
- The TC-PERF timings printed in the test output (for example, availability search times) were measured while branch tracing was active. They are not valid performance evidence and are not used in this report.
- Results apply to commit 61d576d only.

6. Conclusion

At commit 61d576d, the automated suite (131 tests, 0 failures, 2 environment-related skips) executes 117 of 144 branch outcomes in the BBMS application code, which is 81.25%. Accounts and reports are fully covered. Donations (28.6%) and inventory (50.0%) have the lowest branch coverage, and most of their gaps are negative paths: permission denials, invalid forms and invalid input.

The STP sets no branch target, so no pass or fail is assigned. Adding the tests in Section 5.3, particularly one invalid-form test for donations, would close most of the remaining gaps. The possible defect in Section 5.2 should be confirmed by the team before any change is made.

Appendix A. Branch Coverage Terminal Report

Output of python -m coverage report -m for the run described in this report (saved as Evidence/BranchCoverage/branch_summary.txt).

```
Name                                                    Stmts
Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------
-------------------------------------
accounts\__init__.py                                        0
0      0      0   100%
accounts\admin.py                                           6
0      0      0   100%
accounts\apps.py                                            6
0      0      0   100%
accounts\decorators.py                                     14
0      2      0   100%
accounts\models.py                                         11
1      0      0    91%   29
accounts\signals.py                                         8
0      2      0   100%
accounts\urls.py                                            4
0      0      0   100%
accounts\views.py                                           1
1      0      0     0%   1
blood_requests\__init__.py                                  0
0      0      0   100%
blood_requests\admin.py                                     8
0      0      0   100%
blood_requests\apps.py                                      4
0      0      0   100%
blood_requests\forms.py                                    24
0      2      0   100%
blood_requests\models.py                                   33
1      0      0    97%   112
blood_requests\permissions.py                              24
3     10      3    82%   23, 26, 62
blood_requests\services.py                                 12
0      2      0   100%
blood_requests\urls.py                                      4
0      0      0   100%
blood_requests\views.py                                    82
0     30      2    98%   53->56, 112->144
```

```
blood_search\__init__.py                          0
0      0      0   100%
blood_search\admin.py                             1
0      0      0   100%
blood_search\apps.py                              4
0      0      0   100%
blood_search\models.py                            1
0      0      0   100%
blood_search\services.py                          6
0      0      0   100%
blood_search\urls.py                              4
0      0      0   100%
blood_search\views.py                            23
1      8      1    94%   24
donations\__init__.py                             0
0      0      0   100%
donations\admin.py                                1
0      0      0   100%
donations\apps.py                                 4
0      0      0   100%
donations\forms.py                               15
1      2      1    88%   39
donations\models.py                              30
2      2      1    91%   25, 72
donations\urls.py                                 4
0      0      0   100%
donations\views.py                               31
7     10      2    63%   27, 53-61
donors\__init__.py                                0
0      0      0   100%
donors\admin.py                                   1
0      0      0   100%
donors\apps.py                                    4
0      0      0   100%
donors\forms.py                                  10
0      0      0   100%
donors\models.py                                 30
1      0      0    97%   78
donors\urls.py                                    4
0      0      0   100%
donors\views.py                                  49
1     18      3    94%   73->78, 95, 138->156
```

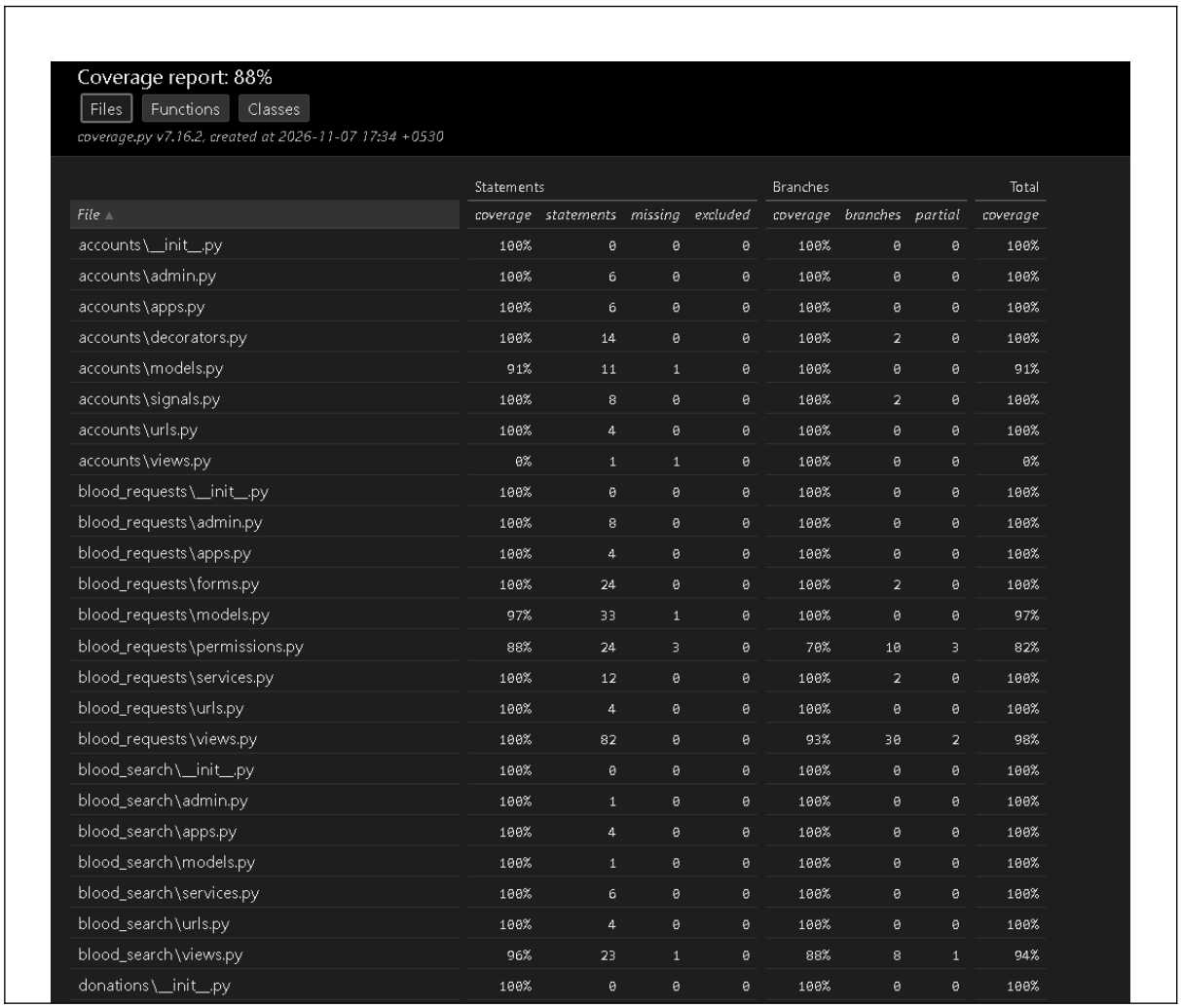
```
inventory\__init__.py                                0
0      0      0   100%
inventory\admin.py                                   1
0      0      0   100%
inventory\apps.py                                    4
0      0      0   100%
inventory\models.py                                  1
0      0      0   100%
inventory\services.py                               28
1      2      1    93%   29
inventory\urls.py                                    4
0      0      0   100%
inventory\views.py                                  13
1      2      1    87%   26
issues\__init__.py                                   0
0      0      0   100%
issues\admin.py                                     23
5      0      0    78%   12, 15, 46, 49, 52
issues\apps.py                                       4
0      0      0   100%
issues\issuance.py                                  39
1     12      2    94%   85->83, 108
issues\management\__init__.py                        0
0      0      0   100%
issues\management\commands\__init__.py               0
0      0      0   100%
issues\management\commands\seed_perf_data.py        57
57      4      0     0%   1-182
issues\models.py                                    28
2      0      0    93%   67, 95
issues\permissions.py                                3
0      0      0   100%
issues\services.py                                   9
0      4      0   100%
issues\urls.py                                       4
0      0      0   100%
issues\views.py                                     56
2     14      0    97%   71-72
monitoring\__init__.py                               0
0      0      0   100%
monitoring\admin.py                                  1
0      0      0   100%
```

```
monitoring\apps.py                                    4
0      0      0   100%
monitoring\models.py                                  1
0      0      0   100%
monitoring\views.py                                   1
1      0      0     0%   1
reports\__init__.py                                   0
0      0      0   100%
reports\admin.py                                      1
0      0      0   100%
reports\apps.py                                       4
0      0      0   100%
reports\forms.py                                     21
0      2      0   100%
reports\models.py                                     1
0      0      0   100%
reports\permissions.py                                3
0      0      0   100%
reports\services.py                                  39
0     12      0   100%
reports\urls.py                                       4
0      0      0   100%
reports\views.py                                     28
0      4      0   100%
------------------------------------------------------------
--------------------------------------
TOTAL                                               845
89    144     17    88%
```

Appendix B. HTML Coverage Report Screenshots

Each screenshot should show the browser title bar, which names the file and its coverage. Captions name the source file so that each image can be matched to the evidence folder.

Figure B1 — coverage.py HTML index, branch run



Coverage report: 88%

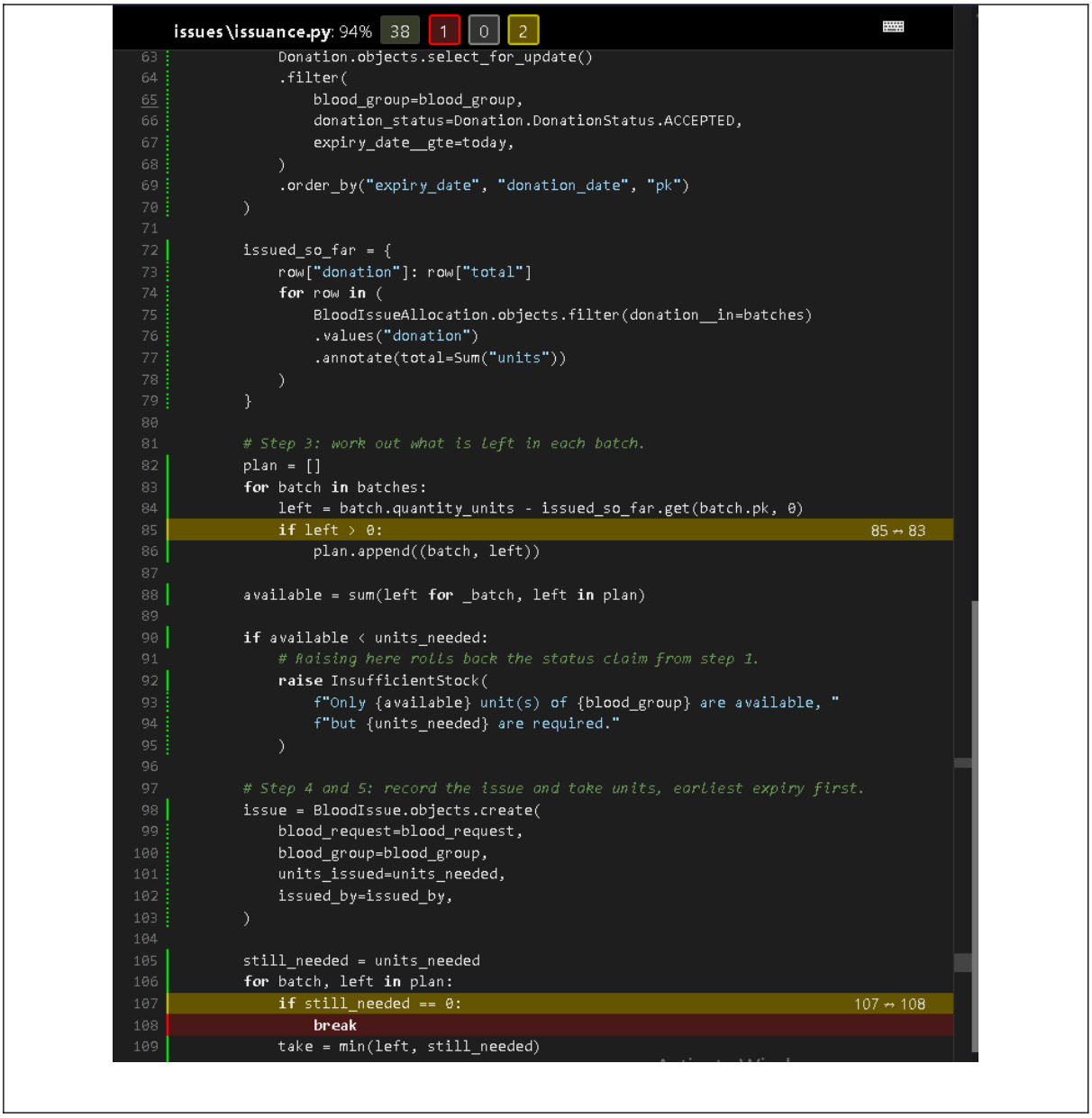
Files | Functions | Classes

coverage.py v7.16.2, created at 2026-11-07 17:34 +0530

	Statements				Branches			Total
File ▲	coverage	statements	missing	excluded	coverage	branches	partial	coverage
accounts__init__.py	100%	0	0	0	100%	0	0	100%
accounts\admin.py	100%	6	0	0	100%	0	0	100%
accounts\apps.py	100%	6	0	0	100%	0	0	100%
accounts\decorators.py	100%	14	0	0	100%	2	0	100%
accounts\models.py	91%	11	1	0	100%	0	0	91%
accounts\signals.py	100%	8	0	0	100%	2	0	100%
accounts\urls.py	100%	4	0	0	100%	0	0	100%
accounts\views.py	0%	1	1	0	100%	0	0	0%
blood_requests__init__.py	100%	0	0	0	100%	0	0	100%
blood_requests\admin.py	100%	8	0	0	100%	0	0	100%
blood_requests\apps.py	100%	4	0	0	100%	0	0	100%
blood_requests\forms.py	100%	24	0	0	100%	2	0	100%
blood_requests\models.py	97%	33	1	0	100%	0	0	97%
blood_requests\permissions.py	88%	24	3	0	70%	10	3	82%
blood_requests\services.py	100%	12	0	0	100%	2	0	100%
blood_requests\urls.py	100%	4	0	0	100%	0	0	100%
blood_requests\views.py	100%	82	0	0	93%	30	2	98%
blood_search__init__.py	100%	0	0	0	100%	0	0	100%
blood_search\admin.py	100%	1	0	0	100%	0	0	100%
blood_search\apps.py	100%	4	0	0	100%	0	0	100%
blood_search\models.py	100%	1	0	0	100%	0	0	100%
blood_search\services.py	100%	6	0	0	100%	0	0	100%
blood_search\urls.py	100%	4	0	0	100%	0	0	100%
blood_search\views.py	96%	23	1	0	88%	8	1	94%
donations__init__.py	100%	0	0	0	100%	0	0	100%

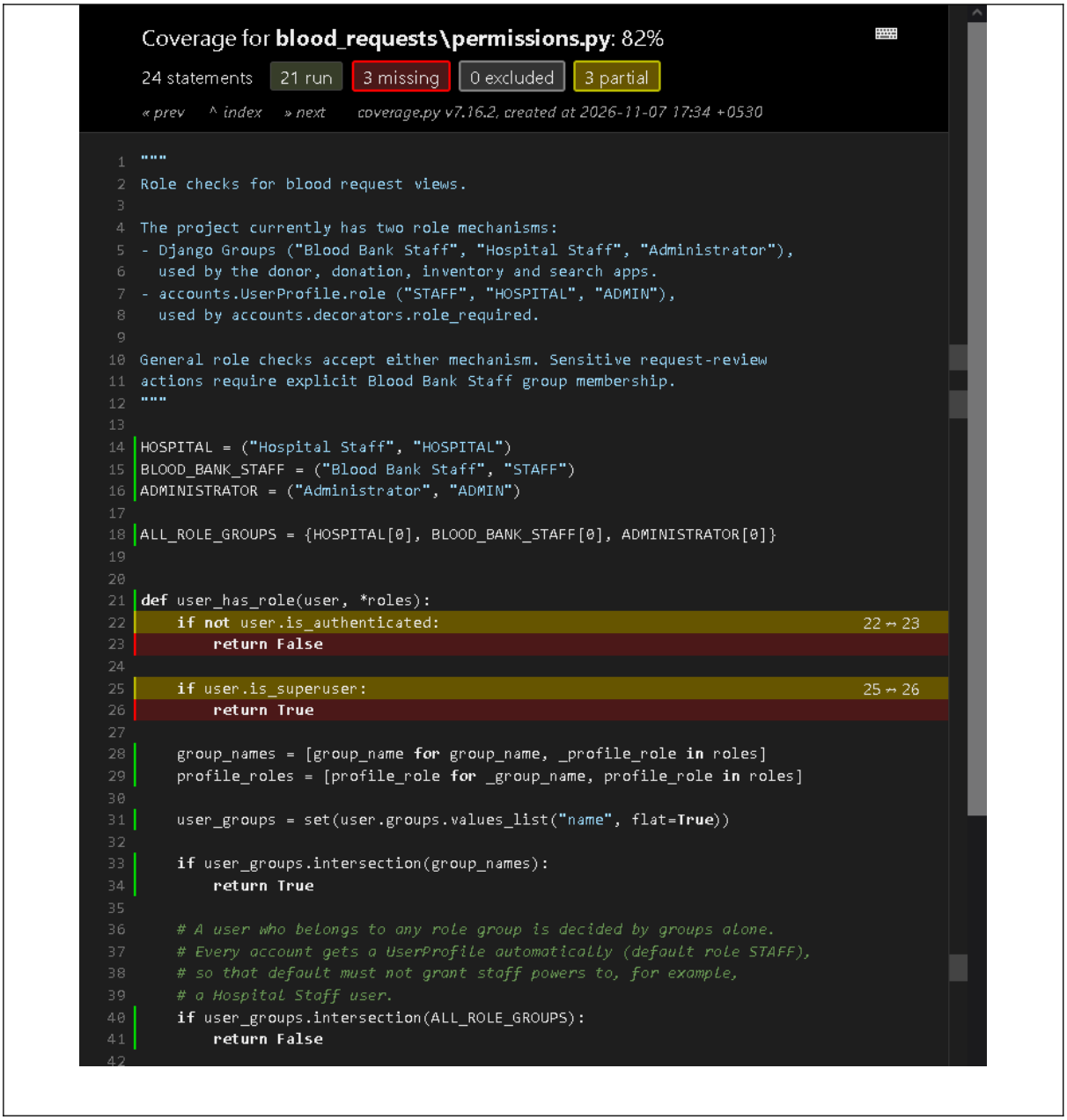
What it shows: Branch and partial columns for every file. The 88% title is the combined score (Section 3.3).

Figure B2 — issues/issuance.py, issue_blood (steps 3–5)



What it shows: Arc 85→83 (exhausted batch skipped) and arc 107→108 (break never reached).

Figure B3 — blood_requests/permissions.py, user_has_role



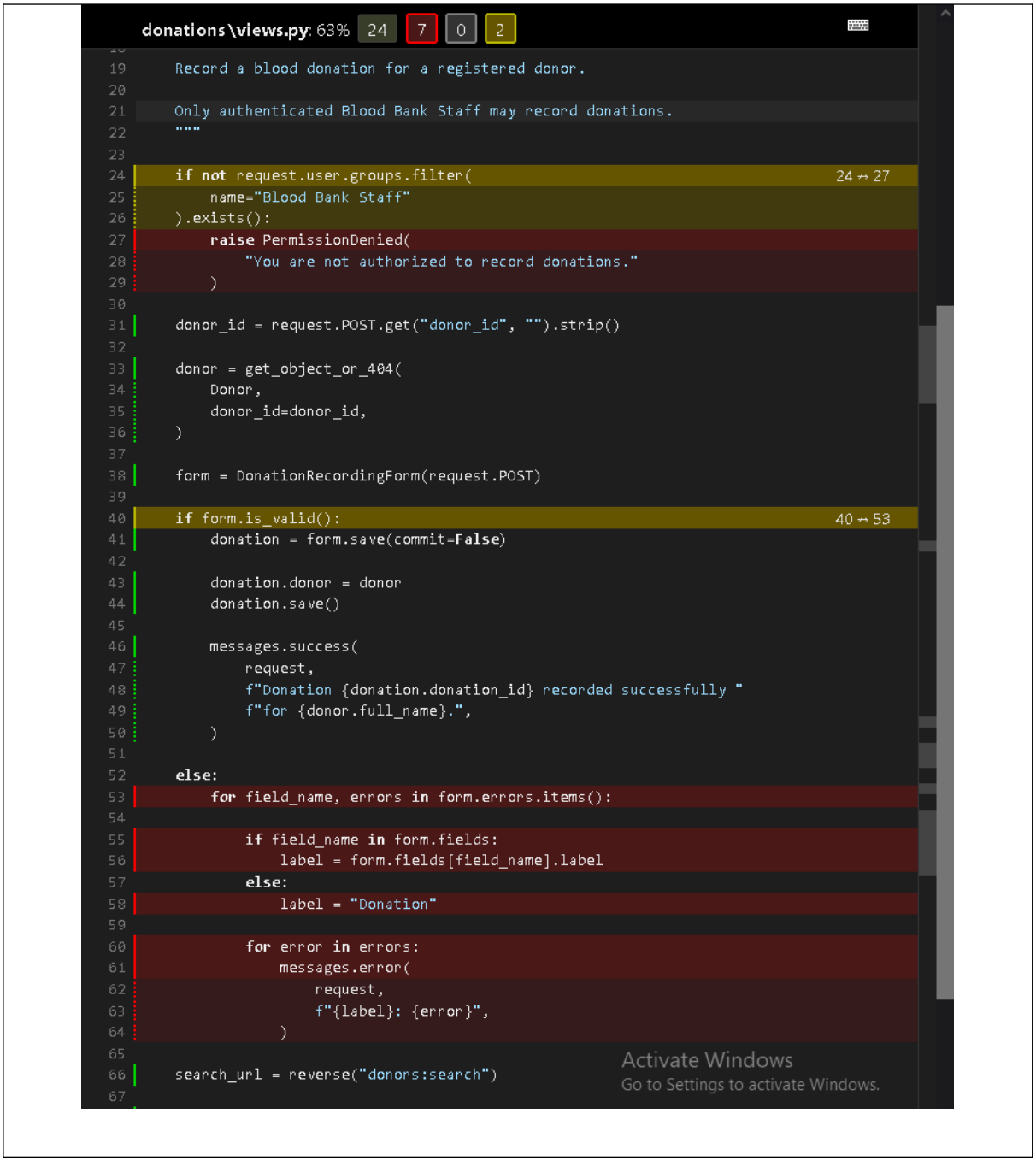
What it shows: Arcs 22→23 (anonymous user) and 25→26 (superuser).

Figure B3b — blood_requests/permissions.py, can_review_request



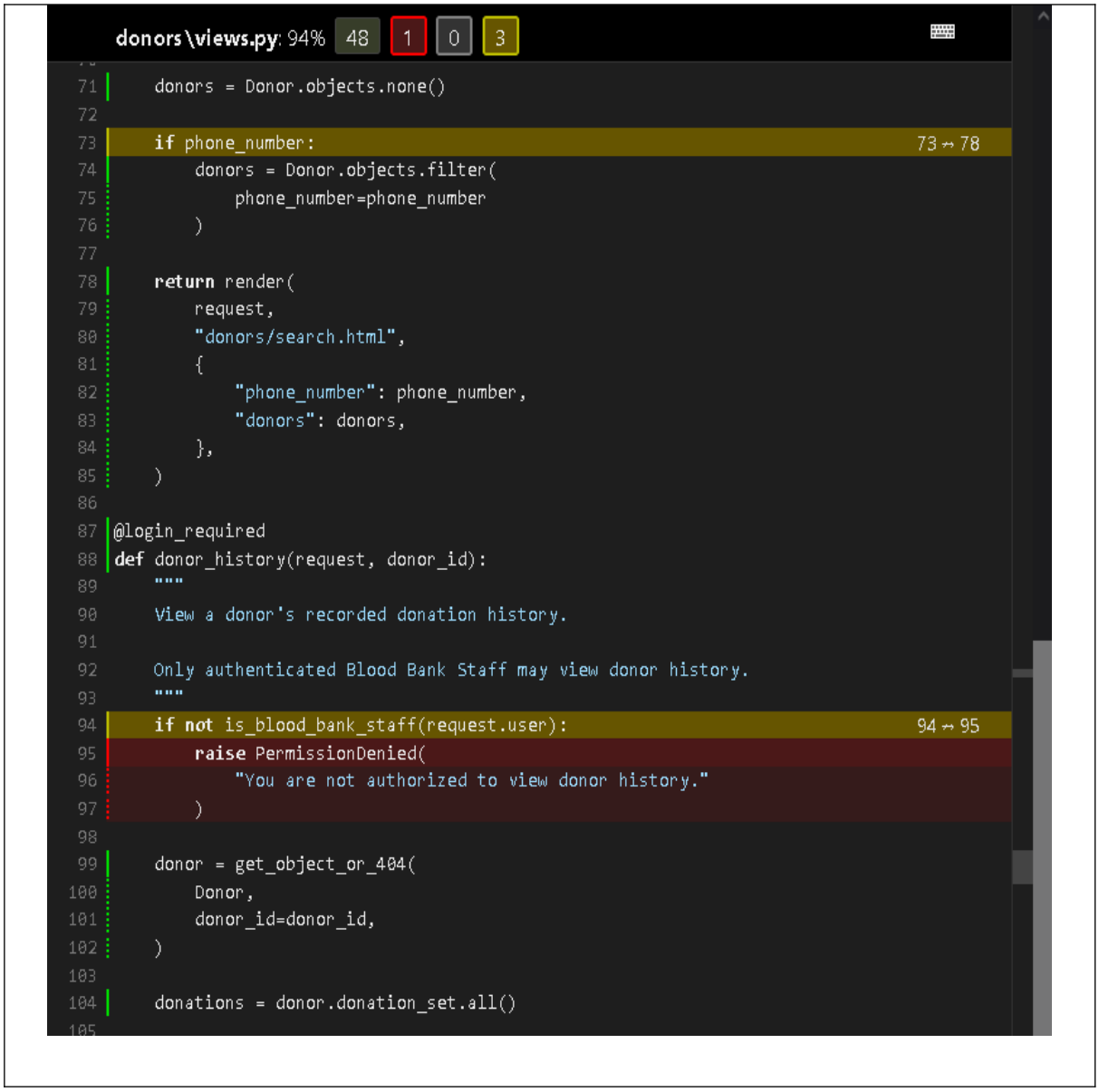
What it shows: Arc 61→62 (anonymous user).

Figure B4 — donations/views.py, record_donation



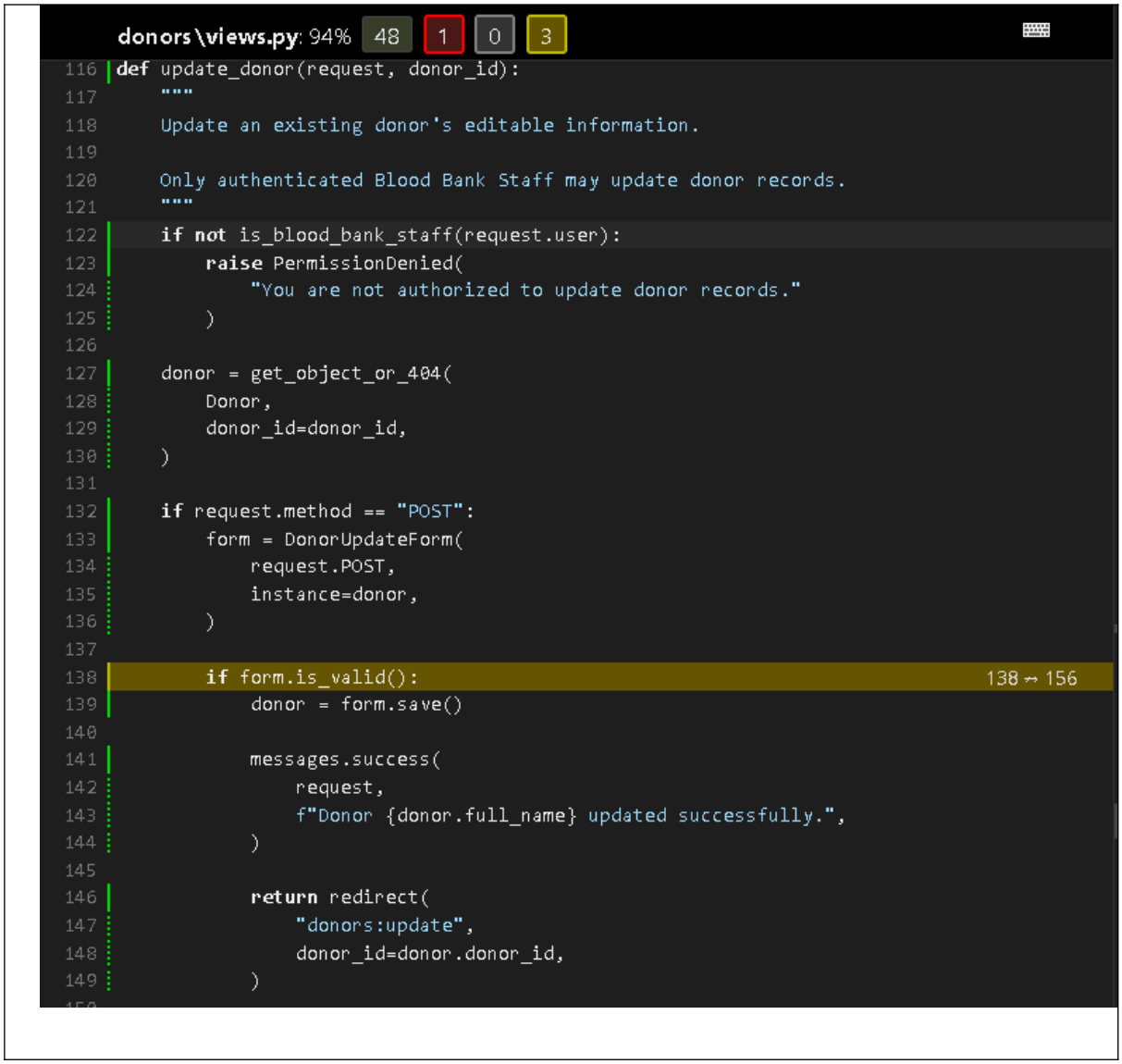
What it shows: Arcs 24→27 (non-staff user) and 40→53 (invalid form), with the form-error block at lines 53–61 not executed.

Figure B5 — donors/views.py, donor search and donor history



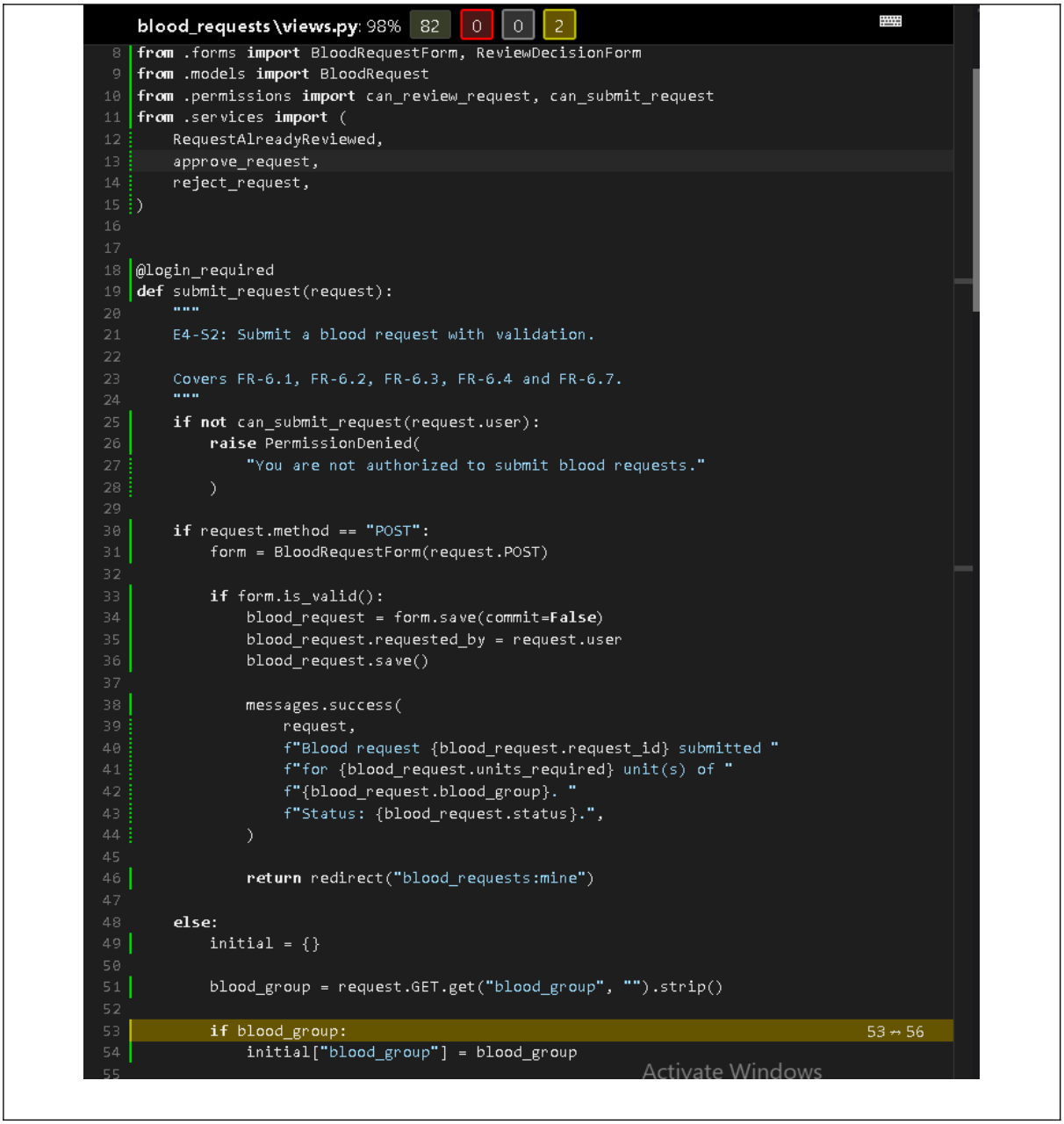
What it shows: Arcs 73→78 (no phone number) and 94→95 (non-staff user).

Figure B5b — donors/views.py, donor update



What it shows: Arc 138→156 (invalid update form).

Figure B6a — blood_requests/views.py, submit view




```
from .forms import BloodRequestForm, ReviewDecisionForm
from .models import BloodRequest
from .permissions import can_review_request, can_submit_request
from .services import (
    RequestAlreadyReviewed,
    approve_request,
    reject_request,
)


@login_required
def submit_request(request):
    """
    E4-S2: Submit a blood request with validation.

    Covers FR-6.1, FR-6.2, FR-6.3, FR-6.4 and FR-6.7.
    """
    if not can_submit_request(request.user):
        raise PermissionDenied(
            "You are not authorized to submit blood requests."
        )

    if request.method == "POST":
        form = BloodRequestForm(request.POST)

        if form.is_valid():
            blood_request = form.save(commit=False)
            blood_request.requested_by = request.user
            blood_request.save()

            messages.success(
                request,
                f"Blood request {blood_request.request_id} submitted "
                f"for {blood_request.units_required} unit(s) of "
                f"{blood_request.blood_group}. "
                f"Status: {blood_request.status}.",
            )

            return redirect("blood_requests:mine")

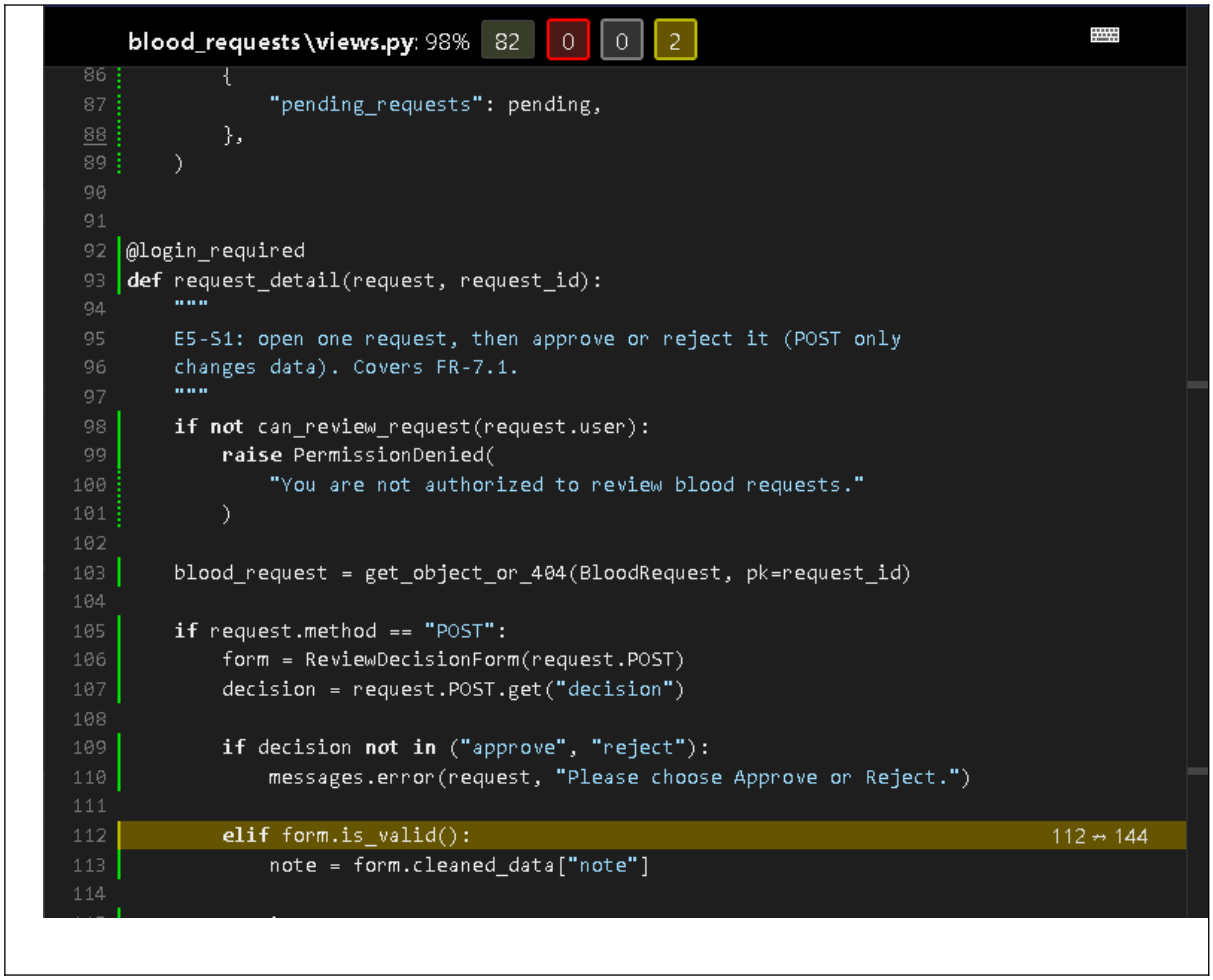
    else:
        initial = {}

        blood_group = request.GET.get("blood_group", "").strip()

        if blood_group:                                    53 ↔ 56
            initial["blood_group"] = blood_group
```

What it shows: Arc 53→56 (no blood_group value).

Figure B6b — blood_requests/views.py, review view



What it shows: Arc 112→144 (valid decision, invalid form; see Section 5.2).

Appendix C. Evidence Files

Files stored in Docs/Testing/Evidence/BranchCoverage/. The full htmlcov/ folder is not committed because it is generated output and is listed in .gitignore; it can be regenerated with the commands in Section 3.1.

File (Docs/Testing/Evidence/BranchCoverage/)	Contents
branch_full_suite.txt	Full test run output with -v 2 (131 tests, OK, skipped=2)
branch_summary.txt	Terminal branch coverage report (Appendix A)
branch_coverage.json	Per-file branch data used for the per-app totals in Section 4.2
B1_coverage_index.png	HTML index, branch columns (Figure B1)
B2_issuance.png	issues/issuance.py partial arcs 85→83 and 107→108 (Figure B2)
B3_permissions.png	blood_requests/permissions.py arcs 22→23 and 25→26 (Figure B3)
B3b_permissions_review.png	blood_requests/permissions.py arc 61→62 (Figure B3b)
B4_donations_views.png	donations/views.py arcs 24→27 and 40→53 (Figure B4)
B5_donors_views.png	donors/views.py arcs 73→78 and 94→95 (Figure B5)
B5b_donors_138.png	donors/views.py arc 138→156 (Figure B5b)
B6a_blood_requests_53.png	blood_requests/views.py arc 53→56 (Figure B6a)
B6b_blood_requests_112.png	blood_requests/views.py arc 112→144 (Figure B6b)